# OTEL-01: OpenTelemetry Fundamentals

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 1 of 8 | **Created:** January 2026 | **Last Updated:** 10/13/2026

## Introduction to OpenTelemetry and Dynatrace

OpenTelemetry (OTel) is the industry-standard framework for collecting telemetry data. Dynatrace fully supports OpenTelemetry through native OTLP ingestion, allowing you to leverage OTel instrumentation while benefiting from Dynatrace's AI-powered analytics — on its own, or combined with OneAgent per workload.

---

## Table of Contents


1. What is OpenTelemetry?
2. OpenTelemetry and OneAgent in Dynatrace
3. Core Components
4. Signal Types: Traces, Metrics, Logs
5. Semantic Conventions
6. Dynatrace OTel Integration
7. Choosing the Combination per Workload


---

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with OTLP ingestion enabled |
| **Permissions** | Platform token: `openpipeline:traces:ingest`, `openpipeline:metrics:ingest`, `openpipeline:logs:ingest`. Classic access token: `openTelemetryTrace.ingest`, `metrics.ingest`, `logs.ingest` |
| **Knowledge** | Basic observability concepts |

## 1. What is OpenTelemetry?

OpenTelemetry is a vendor-neutral, open-source observability framework that provides:

| Component | Purpose |
|---|---|
| **APIs** | Standardized interfaces for instrumentation |
| **SDKs** | Language-specific implementations |
| **Collector** | Agent for receiving, processing, and exporting telemetry |
| **Protocol (OTLP)** | Standard wire protocol for telemetry data |

### CNCF Project

OpenTelemetry is a Cloud Native Computing Foundation (CNCF) project, formed from the merger of OpenTracing and OpenCensus.

### Key Benefits

| Benefit | Description |
|---|---|
| **Vendor Neutral** | Open standard — the same instrumentation works with any OTLP-compatible backend |
| **Standardized** | Consistent across languages and platforms |
| **Comprehensive** | Traces, metrics, and logs in one framework |
| **Community-Driven** | Active development, broad adoption |

## 2. OpenTelemetry and OneAgent in Dynatrace

OpenTelemetry and OneAgent are complementary, not alternatives. OpenTelemetry is the open, vendor-neutral foundation: one instrumentation standard and common semantic conventions across languages, environments and tools. OneAgent adds automation and runtime depth on top of that foundation. Dynatrace analyzes the two together *"regardless of how you mix your instrumentation choices"*, so the choice is made per workload rather than once for the whole organization.

### What Each Adds

| Aspect | OpenTelemetry | Dynatrace OneAgent |
|---|---|---|
| **Role** | Open telemetry foundation — APIs, SDKs, semantic conventions, OTLP | Automation and runtime depth on top of that foundation |
| **Instrumentation** | Per-runtime auto-instrumentation, plus SDK calls for custom spans | Automatic — injected at process start, no code change |
| **Deployment** | SDK and/or Collector, owned by your teams | Single agent with a centrally managed lifecycle |
| **Coverage** | The signals you choose to emit | Full stack — host, process, runtime, code level, topology |
| **Portability** | Any OTLP-compatible backend, Dynatrace included | Dynatrace, where its data is analyzed alongside OTel data |
| **Dynatrace Intelligence** | Yes — including service detection and service topology from OTel spans | Yes — with infrastructure topology filled in automatically |
| **Runtime-access capabilities** | Not provided by OTel telemetry | Application Security, Live Debugger, automatic RUM JavaScript injection |

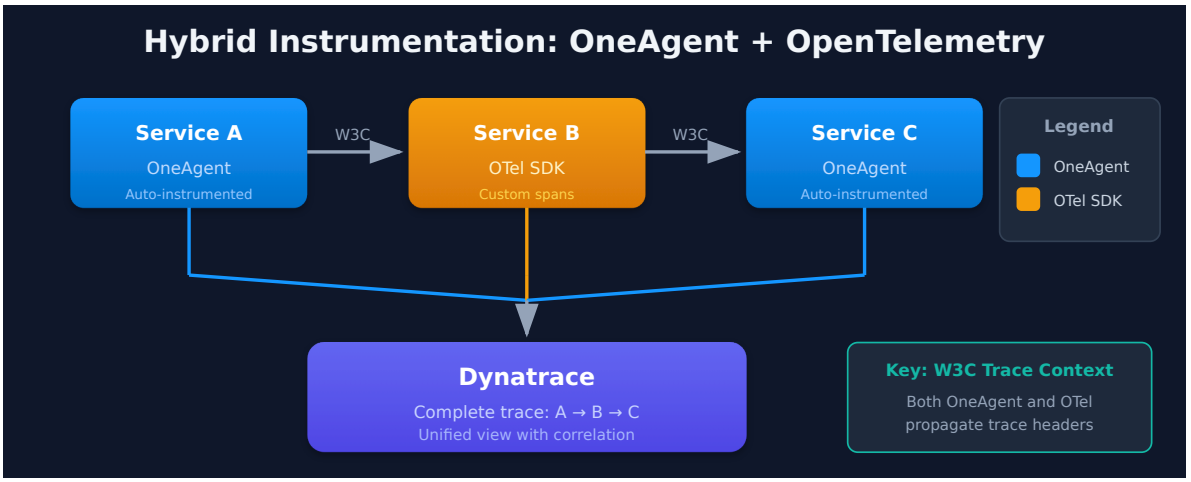
### Choosing per Workload

| Scenario | Recommendation |
|---|---|
| Organization has chosen OpenTelemetry | OTel — a complete path; add OneAgent only where its automation or runtime-access capabilities are wanted |
| No preference, standard runtimes on hosts, VMs or Kubernetes | OneAgent — fully managed, lowest effort; OTel for custom business spans |
| Existing OTel instrumentation | Keep it; add OneAgent alongside where runtime depth is wanted (see FAQ-03) |
| Prometheus endpoints | OTel Collector — the documented recommendation for new deployments |

| Scenario | Recommendation |
| --- | --- |
| Open-standard or multi-backend instrumentation mandate | OTel owns application instrumentation; OneAgent alongside where it runs |
| AWS Lambda | Dynatrace Lambda layer (Python, Node.js, Java, .NET); OTel for custom spans |
| Azure Functions | OneAgent on the plans and runtimes OneAgent 1.343+ supports; OTel elsewhere |
| GCP Cloud Functions, Ruby application code | OTel for application spans; OneAgent for host and process where it runs |
| Application Security, Live Debugger or RUM auto-injection needed | OneAgent on those workloads |
| Minimal effort | OneAgent |

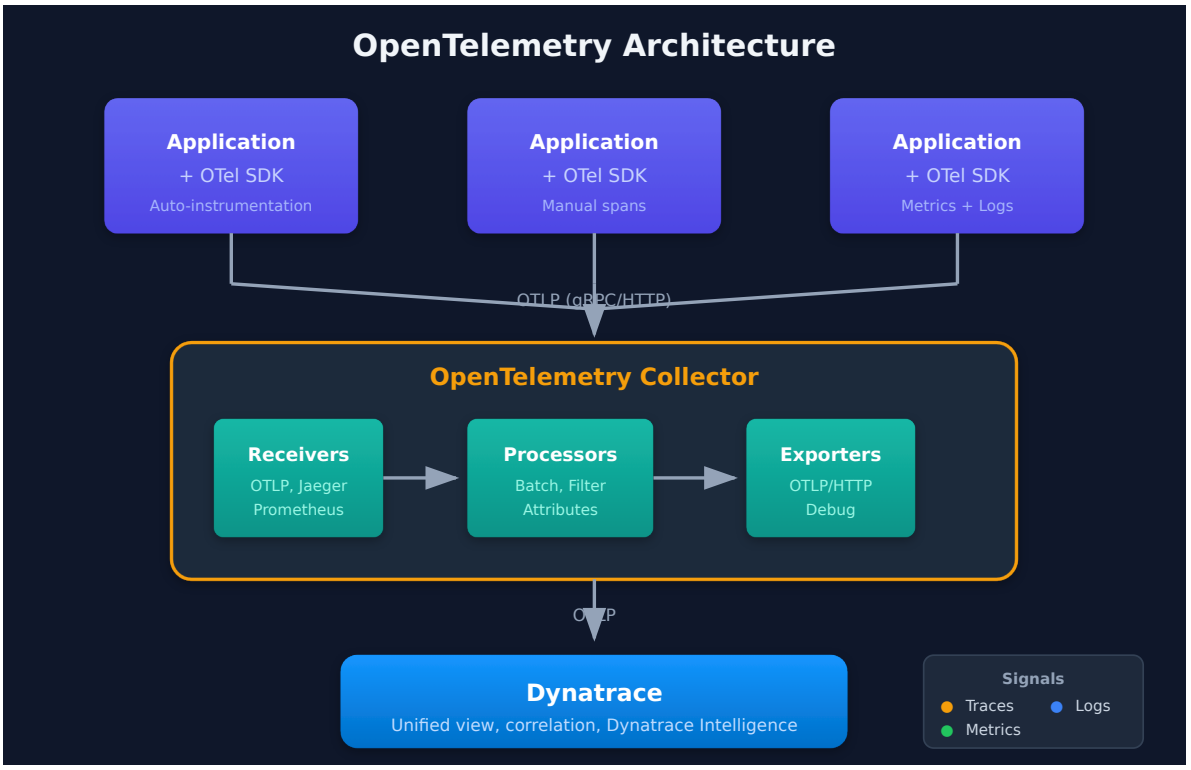
## Using Both Together

Dynatrace supports using OneAgent and OTel together — in the same application, or across teams and workloads in the same environment:



## 3. Core Components

### OpenTelemetry Architecture




### Component Descriptions

| Component | Role |
|---|---|
| **API** | Defines how to instrument (language-specific interfaces) |
| **SDK** | Implements the API, handles sampling, batching |
| **Exporter** | Sends data to backends (OTLP, Jaeger, etc.) |
| **Collector** | Standalone service for processing telemetry |
| **Instrumentation Libraries** | Pre-built instrumentation for frameworks |

## 4. Signal Types: Traces, Metrics, Logs

### Traces

Distributed traces track requests across services.

| Concept | Description |
|---|---|
| **Trace** | End-to-end transaction path |
| **Span** | Single operation within a trace |
| **SpanContext** | Propagated context (trace ID, span ID) |
| **Attributes** | Key-value metadata on spans |

| Concept | Description |
|---|---|
| **Events** | Timestamped annotations on spans |

### Metrics

Quantitative measurements over time.

| Instrument Type | Use Case | Example |
|---|---|---|
| **Counter** | Monotonically increasing value | Request count |
| **UpDownCounter** | Value that goes up and down | Active connections |
| **Histogram** | Distribution of values | Response time |
| **Gauge** | Current value | Temperature, queue size |

### Logs

Structured event records correlated with traces.

| Field | Description |
|---|---|
| **Timestamp** | When the event occurred |
| **Severity** | Log level (DEBUG, INFO, ERROR) |
| **Body** | Log message content |
| **Attributes** | Structured metadata |
| **TraceContext** | Link to active span |

## 5. Semantic Conventions

Semantic conventions define standard attribute names for consistent telemetry.

> **Note:** OpenTelemetry semantic conventions are evolving. HTTP conventions were **stabilized in 2023** with new names (see migration tables below). Database conventions are also migrating. SDKs support a gradual transition via the `OTEL_SEMCONV_STABILITY_OPT_IN` environment variable. Both old and new names may coexist during migration.

### Resource Attributes

| Attribute | Example | Description |
|---|---|---|
| `service.name` | `checkout-api` | Logical service name |
| `service.version` | `1.2.3` | Service version |
| `service.namespace` | `shop` | Groups services (team, system or application); scopes `service.name` |
| `deployment.environment.name` | `prod` | Deployment environment |

> **Note:** `deployment.environment` was renamed to `deployment.environment.name` in the stable resource semantic conventions.

### HTTP Span Attributes

| Old (Legacy) | Stable | Description |
|---|---|---|
| `http.method` | `http.request.method` | HTTP method |
| `http.url` | `url.full` | Full URL |
| `http.status_code` | `http.response.status_code` | Response status code |
| `http.route` | `http.route` (unchanged) | Route template |
| `http.target` | `url.path` + `url.query` | Request target |
| `http.host` | `server.address` | Server hostname |

> **Tip:** Use `OTEL_SEMCONV_STABILITY_OPT_IN=http` to emit only stable names, or `http/dup` to emit both old and new during migration. See the HTTP semconv migration guide.

### Database Span Attributes

| Old (Legacy) | Stable | Description |
|---|---|---|
| `db.name` | `db.namespace` | Database name/namespace |
| `db.statement` | `db.query.text` | Query statement |
| `db.operation` | `db.operation.name` | Operation type |
| `db.system` | `db.system` | Database type (unchanged) |

> **Tip:** Use `OTEL_SEMCONV_STABILITY_OPT_IN=database` for stable database names. See the DB semconv migration guide.

### Kubernetes Attributes

| Attribute | Example | Description |
|---|---|---|
| `k8s.namespace.name` | `checkout` | Namespace |
| `k8s.pod.name` | `checkout-api-abc123` | Pod name |
| `k8s.deployment.name` | `checkout-api` | Deployment |

## 6. Dynatrace OTel Integration

### OTLP Endpoints

Dynatrace accepts OTLP data natively via **HTTP only**, with binary Protocol Buffers: *"gRPC is not supported. API calls need to use HTTP."* and *"JSON is not supported for Protocol Buffers. Binary format must be used."*

| Protocol | Endpoint | Notes |
|---|---|---|
| **HTTP** | `https://{your-env}.live.dynatrace.com/api/v2/otlp` | Recommended for direct ingest |
| **gRPC** | Not supported for direct Dynatrace ingest | Use a Collector to convert gRPC → HTTP |

> **Important:** If your application uses gRPC to export OTLP data, route it through an OpenTelemetry Collector configured with an `otlp` gRPC receiver and an `otlphttp` exporter to Dynatrace. See Transform OTLP gRPC.

## Authentication

Two token types work, with different schemes and scopes:

| Token | Header | Traces | Metrics | Logs |
|---|---|---|---|---|
| Platform token | `Authorization: Bearer <token>` | `openpipeline:traces:ingest` | `openpipeline:metrics:ingest` | `openpipeline:logs:ingest` |
| Classic access token | `Authorization: Api-Token <token>` | `openTelemetryTrace.ingest` | `metrics.ingest` | `logs.ingest` |

> **Sources:** OTLP API (DT docs) — *"Platform token: Use Bearer in the Authorization header. The required scopes are openpipeline:logs:ingest, openpipeline:metrics:ingest, and openpipeline:traces:ingest."*

## Configuration Example

**OTel Collector Exporter:**

```
exporters:
  otlphttp:
    endpoint: https://{your-env}.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: Api-Token ${DT_API_TOKEN}
```

**SDK Direct Export (Python):**

```
import os

from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

exporter = OTLPSpanExporter(
    endpoint="https://{your-env}.live.dynatrace.com/api/v2/otlp/v1/traces",
    headers={"Authorization": f"Api-Token {os.environ['DT_API_TOKEN']}"}
)
```

## OpenTelemetry Licensing & Cost

OpenTelemetry data is billed exactly like any other ingested data under the Dynatrace Platform Subscription (DPS) — there is no separate OTel licence and no OTel surcharge or discount. Each signal maps to DPS capabilities:

| Signal | Ingest & Process | Retain | Query |
|---|---|---|---|
| Traces | per GiB ingested | GiB-days (10 days included) | per GiB scanned (Distributed Tracing / Services apps don't bill queries) |
| Metrics | per 100k data points | GiB-days (15 months @ 1-min granularity included) | included with Ingest & Process |
| Logs | per GiB ingested | GiB-days | per GiB scanned |

**Included volume follows the source, not the protocol:** Full-Stack Monitoring includes a defined trace volume *"for traces that are sent via the OneAgent Trace API or from a Full-Stack monitored host"*, and *"Metrics originating from Full-Stack-monitored*

*hosts or containers include a defined amount of metric data points"* — you are charged only for data above the included amount. OTel telemetry from sources outside Full-Stack Monitoring (for example, a workload on a host with no OneAgent) has no included volume to draw on and bills on the rate card as ingested. Practical consequence: high-volume OTel traces/metrics from such sources are a direct cost line — size and (if needed) sample them deliberately (see OTEL-04 §7 and the FINOPS series).

> **Sources:** OpenTelemetry licensing (DT docs) — *"With Dynatrace, OpenTelemetry data is treated like any other ingested data."* **Derived:** OTel data from sources outside Full-Stack Monitoring has no included volume, because the page scopes the allowance to trace and metric data from those sources.

```
// View OpenTelemetry traces that arrived over OTLP
// dt.openpipeline.source identifies the ingest path. otel.scope.name is NOT a reliable
// "is OTel" test: OneAgent spans can carry an instrumentation scope too.
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| fields start_time, trace.id, service.name, span.name, otel.scope.name, duration
| sort start_time desc
| limit 20
```

```
// OTel instrumentation libraries in use (OTLP-ingested spans)
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| summarize count = count(), by:{otel.scope.name, otel.scope.version}
| sort count desc
| limit 20
```

## 7. Choosing the Combination per Workload

### Where OpenTelemetry Leads

| Scenario | Why OTel |
|---|---|
| **OTel as the collection strategy** | A complete path — services, endpoints and service topology are detected from OTel data |
| **Custom business telemetry** | Spans and metrics specific to your domain |
| **Prometheus endpoints** | Collector-based scraping is the documented recommendation for new deployments |
| **Open-standard mandate** | One instrumentation model across teams and backends |
| **Multi-cloud / polyglot** | Consistent instrumentation across clouds and languages |
| **Where OneAgent does not run** | GCP Cloud Functions, unsupported Azure Functions plans, Ruby application code |
| **Edge/IoT** | Lightweight collection |

### Where OneAgent Adds Depth

| Scenario | Why OneAgent |
|---|---|
| **Full-stack** | Infrastructure, process, runtime and code in one install |
| **Zero-effort** | Automatic instrumentation, centrally managed updates |
| **Infrastructure topology** | Hosts, processes and containers in Smartscape without running a Collector |
| **PurePath** | Code-level transaction tracing |
| **Application Security** | Runtime Vulnerability Analytics and Runtime Application Protection |
| **Live Debugger** | Java and Node.js, OneAgent 1.309+ |

| Scenario | Why OneAgent |
|---|---|
| **Real User Monitoring** | Automatic RUM JavaScript injection |
| **Serverless (supported platforms)** | AWS Lambda layer; supported Azure Functions plans from OneAgent 1.343 |

### Recommended: Both, per Workload

For most enterprises:
- **OTel-only is a complete path** where OTel is the collection strategy — the tooling decision should not push teams off it
- **OneAgent** where teams want fully managed collection and the capabilities that need runtime access
- **Both** where a workload benefits from OTel's open instrumentation and OneAgent's depth together
- **Mixed estates are normal** — decide per team, application, host or Kubernetes workload

## Next Steps

Now that you understand OTel fundamentals, proceed to:

| Next Notebook | Topic |
|---|---|
| **OTEL-02: Collector Architecture** | Deep-dive into the Collector |
| **OTEL-03: Collector Deployment** | Deployment patterns |
| **OTEL-04: Trace Instrumentation** | Instrumenting your code |

---

## Summary

In this notebook, you learned:

- What OpenTelemetry is and its benefits
- How OpenTelemetry and OneAgent complement each other
- Core OTel components: API, SDK, Collector
- Signal types: traces, metrics, logs
- Semantic conventions for consistent telemetry (including the stable HTTP and DB naming migration)
- Dynatrace OTLP integration configuration (HTTP only — gRPC requires Collector)
- How to choose the OTel and OneAgent combination per workload

---

## References


- OpenTelemetry Official Docs
- Dynatrace OpenTelemetry Integration — *"regardless of how you mix your instrumentation choices"*
- Dynatrace OpenTelemetry Collector (Dynatrace blog, 03/18/2024) — *"Used together, Dynatrace extends OpenTelemetry observability, and OpenTelemetry extends Dynatrace observability."*
- Unified services (DT docs) — services detected with Service Detection v2 *"were first introduced for OpenTelemetry"*
- OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
- Prometheus data source (DT docs) — *"For high-volume Prometheus scraping in Kubernetes, and for new deployments, consider the OpenTelemetry Collector"*

- Runtime Vulnerability Analytics (DT docs) — *"Dynatrace OneAgent monitors loaded libraries, runtime components, and data flows in real time."*
- Runtime Application Protection (DT docs) — *"Code-level visibility provided by OneAgent"*
- Live Debugger (DT docs) — *"OneAgent version 1.309+ with Java Live-Debugger and Node.js Live-Debugger"*
- RUM JavaScript injection (DT docs) — *"Once OneAgent is deployed in full-stack monitoring mode on a host, it by default automatically injects the RUM JavaScript"*
- AWS Lambda integration (DT docs) — *"using auto-instrumentation without code changes"*
- OneAgent 1.343 release notes (DT docs) — Azure Functions: Python on the Flex Consumption Plan for Linux; Java and Node.js on Consumption, Premium, and Dedicated plans for Windows
- OTLP Specification
- Semantic Conventions
- HTTP Semconv Migration Guide
- DB Semconv Migration Guide

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace.*

*Always verify information against official Dynatrace documentation.*